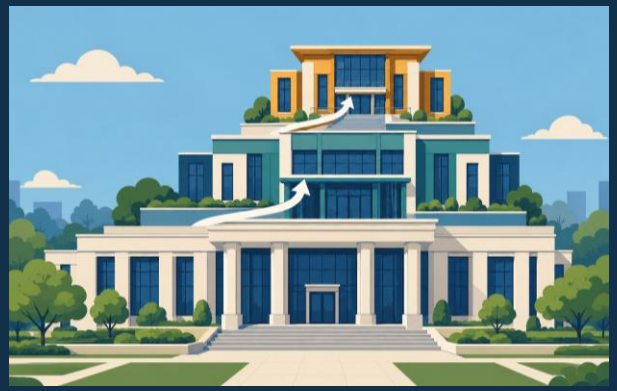
CSC241 / BAI / LECTURE 14

# Constructor Chaining, Protected Access & final

Completing the first inheritance hierarchy safely



Dr Muhammad Shahid Bhatti
COMSATS University Islamabad, Lahore Campus

# Today’s scope follows the CDF

### Lecture 13

Introduction to inheritance and subclasses
The super keyword

### Lecture 14

Constructor chaining
Protected access specifier
The final modifier

**Today continues the same Person → Student hierarchy.**

# Learning outcomes

## By the end, you can

Trace constructor calls through a multi-level hierarchy.

Explain implicit and explicit superclass constructor calls.

Use protected with the correct access boundary.

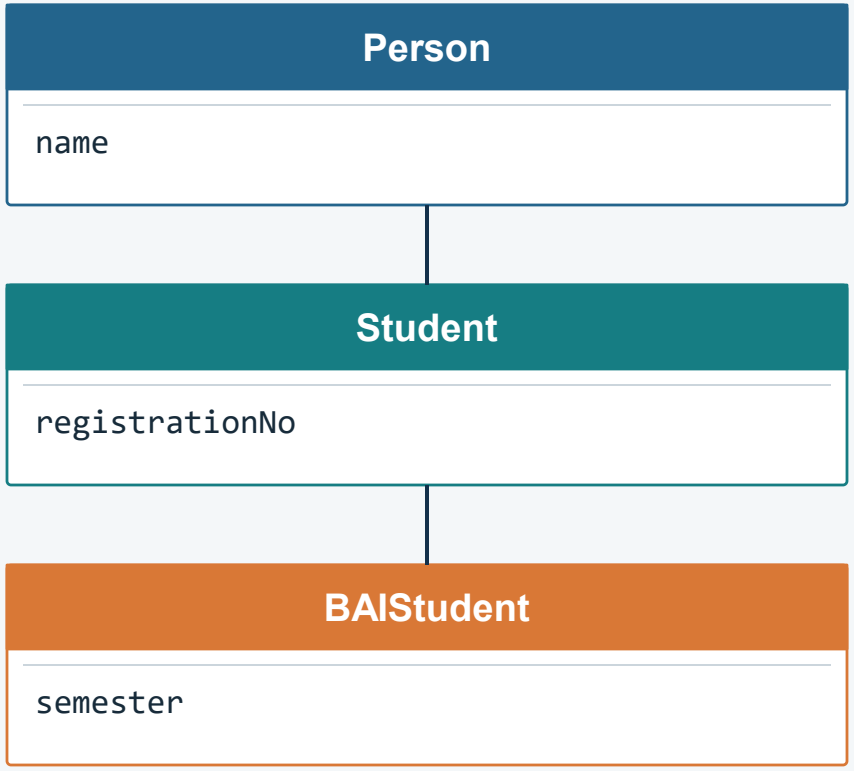
## You will also be able to

Distinguish this(...) from super(...).

Apply final to variables, methods, and classes.

Diagnose common compiler errors.

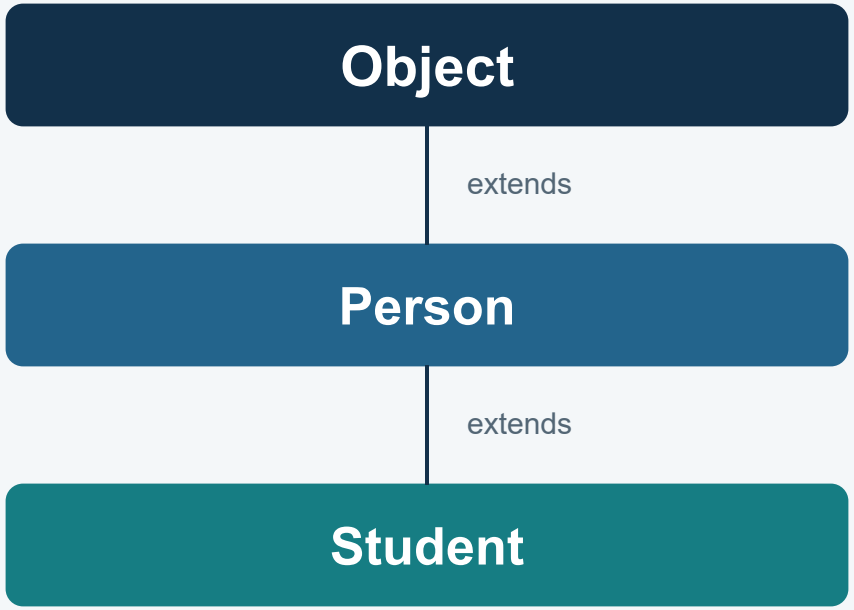
# A subclass object contains state from its ancestors






The object is created once, but each class initializes its own part.

# Every Java class hierarchy reaches Object




If a class does not write extends, Java gives it Object as its direct superclass. A longer chain still ends at Object.

This is a short inheritance bridge; Object methods are treated in Lecture 16.

# toString() is inherited from Object

```
Person p = new Person();
System.out.println(p);
```

println(Object) asks the object for text through toString().

## What to remember now

- If Person does not override it, the inherited Object.toString() supplies a diagnostic representation.
- The default form is not a polished display string or stable identifier.
- Classes can override toString() to show useful field values.

**Lecture 16 develops Object and toString() in detail.**

# Constructors are not inherited

## A constructor belongs to its class

Person(...) constructs Person state.

Student(...) constructs Student state and arranges for Person state to be initialized.

## What inheritance gives you

Accessible fields and methods follow Java access rules.

A subclass must declare its own constructors when it needs specific creation options.

# super(...) selects a superclass constructor

```
public class Student extends Person {
    private String registrationNo;

    public Student(String name, String regNo) {
        super(name);
        this.registrationNo = regNo;
    }
}
```

**KEY IDEA**

super(name) invokes Person(String).

It must be the first statement in the constructor.

Then Student initializes its own field.

# Construction proceeds from superclass to subclass

```
new Student("Abeer", "CIIT/SP26-BAI-005/LHR")
```

1. Person constructor
2. Student constructor
3. **Student object is ready**

# An omitted super() call is inserted by Java

```
// Person.java
public class Person {
    public Person() {
        System.out.println("Person");
    }
}

// Student.java
public class Student extends Person {
    public Student() {
        // Java inserts super(); here
        System.out.println("Student");
    }
}
```

**KEY IDEA**

When the constructor has no explicit this(...) or super(...), Java inserts super().

This works only if the direct superclass has an accessible no-argument constructor.

# The implicit call fails when no no-argument constructor exists

### Superclass requires a value

```
class Person {
    Person(String name) { ... }
}
```

### Subclass omits super(name)

```
class Student extends Person {
    Student() { ... }
}
```

**Java tries super(), but Person() does not exist.**

Fix: call an available constructor explicitly, such as super(name).

# this(...) and super(...) choose different constructors

### this(...)

Calls another constructor in the same class.

Useful for sharing initialization among overloaded constructors.

### super(...)

Calls a constructor in the direct superclass.

Useful for initializing inherited state.

Either call must be the first constructor statement, so one constructor cannot begin with both.

# this(...) can delegate within one class

```
public class Student extends Person {
    private String registrationNo;

    public Student(String name) {
        this(name, "UNASSIGNED");
    }

    public Student(String name, String regNo) {
        super(name);
        this.registrationNo = regNo;
    }
}
```

**KEY IDEA**

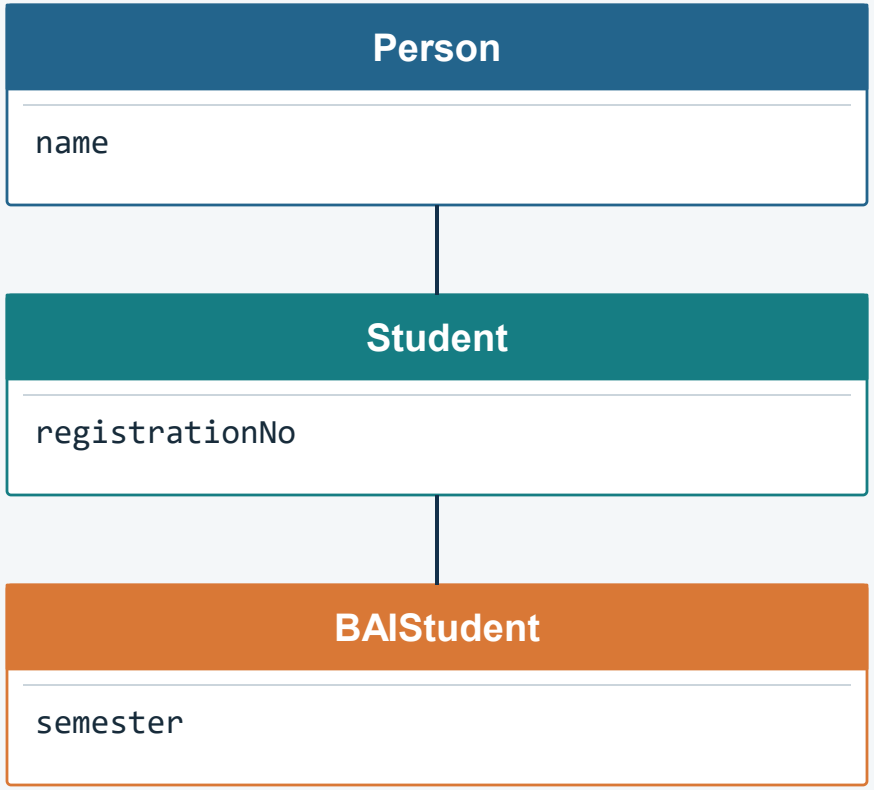
Student(String) delegates to Student(String, String).

The target constructor then calls super(name).

A constructor using this(...) cannot also call super(...).



# A three-level chain initializes each level once




BAIStudent(...) calls Student(...), which calls Person(...).

# Each constructor passes the data it owns

```
// Person.java
public class Person {
    private String name;
    public Person(String name) { this.name = name; }
}

// Student.java
public class Student extends Person {
    private String registrationNo;
    public Student(String name, String regNo) {
        super(name);
        this.registrationNo = regNo;
    }
}
```

**KEY IDEA**

Each class owns its own fields.

Student does not assign Person.name directly.

Each public class belongs in its matching .java file.

# The third constructor continues the chain

```
// BAIStudent.java
public class BAIStudent extends Student {
    private int semester;

    public BAIStudent(String name, String regNo,
                      int semester) {
        super(name, regNo);
        this.semester = semester;
    }
}
```

**KEY IDEA**

super(name, regNo) selects Student(String, String).

Student then calls Person(String).

The constructors return in reverse order after each body finishes.

# Predict the constructor output

If each constructor prints its class name:

| | |
|---|---|
| **Person** | first |
| **Student** | second |
| **BAIStudent** | third |

**Why does Java produce this order?**

# Protected access extends beyond private

### private

Visible only inside its declaring class. Subclass code cannot directly access it.

### protected

Visible within the same package. Also accessible to subclasses in other packages, subject to Java's access rules.



**protected is more open than private, but it does not mean unrestricted public access.**

# Access modifiers define who can use a member

| Modifier | Same class | Same package | Subclass / other package |
|---|---|---|---|
| **private** | Yes | No | No |
| **package-private** | Yes | Yes | No |
| **protected** | Yes | Yes | Yes* |
| **public** | Yes | Yes | Yes |

*Across packages, protected access is available through subclass context, not as unrestricted access through any superclass object.

# A subclass can use a protected superclass field

```
// Person.java
public class Person {
    protected String name;
    public Person(String name) { this.name = name; }
}

// Student.java
public class Student extends Person {
    public Student(String name) { super(name); }
    public void printName() {
        System.out.println(name);
    }
}
```

**KEY IDEA**

Student inherits the protected name field.

For simple teaching examples, this works when both classes share a package.

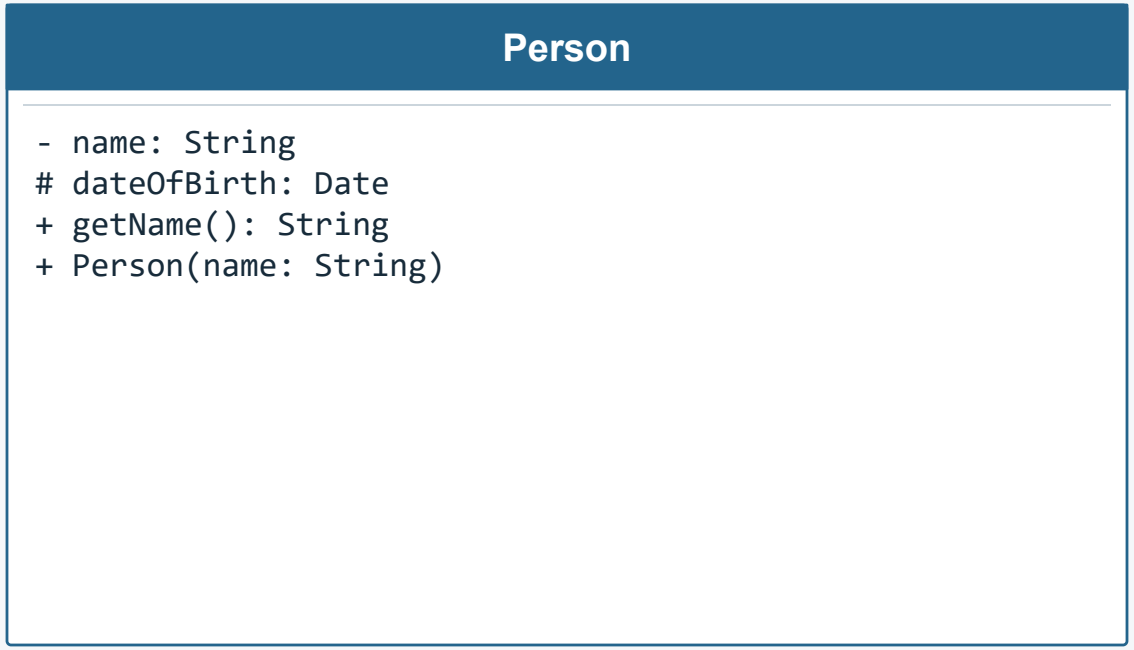
Protected access should be used deliberately.

# UML marks protected members with #

- private

# protected

+ public




Access symbols let readers see the class boundary before reading the code.

# final can restrict reassignment or inheritance

## final variable

Assign once

## final method

Cannot be overridden

## final class

Cannot be extended



**The effect depends on what final modifies.**

# A final field can hold an assigned-once value

```
public class Student extends Person {
    private final String registrationNo;

    public Student(String name, String regNo) {
        super(name);
        registrationNo = regNo;
    }
}
```

**KEY IDEA**

The constructor assigns registrationNo once.

Later code cannot assign a different String reference to that field.

A final field must receive a value before construction completes.

# A final method cannot be overridden

```
// Person.java
public class Person {
    public final String getUniversity() {
        return "COMSATS University Islamabad";
    }
}

// Student.java
public class Student extends Person {
    // Cannot override getUniversity()
}
```

**KEY IDEA**

Person keeps this behavior fixed for subclasses.

A same-signature method in Student would cause a compile-time error.

Use final only when the design needs that restriction.

# A final class cannot have subclasses

```
public final class StudentRecord
{
    // fields and methods
}
```

```
public class BAIRecord
        extends StudentRecord {
    // compile-time error
}
```

**A final class closes the inheritance path at that class.**

# A final reference does not make its object immutable

## Reference cannot change

final Date examDate = new Date(...);

examDate = anotherDate;  // error

## Object may still change

If Date has setters, this may compile:

examDate.setYear(2026);

final protects the reference assignment, not the object state.

# Compile-error clinic

| | |
|---|---|
| **Student constructor omits super(name)** | Java inserts super(); but Person() does not exist. |
| **super(name) appears after an assignment** | The superclass call must be first. |
| **Constructor starts with this(...) and super(...)** | Choose one first-call path; the delegated constructor can call super. |
| **Subclass overrides a final method** | Remove the override or redesign the superclass rule. |
| **Class extends a final class** | A final class blocks subclassing. |

# Practice: complete the constructor chain

**Person(name) ← Student(name, regNo) ← BAIStudent(name, regNo, semester)**

1. Write one constructor for each class.
2. Place each super(...) call correctly.
3. Add a protected field that a subclass can read.
4. Mark registrationNo final and assign it once.
5. Predict constructor output before running Demo.

# Exit ticket

1. Which constructor runs first in BAIStudent → Student → Person?
2. When does Java insert super()?
3. What is the key difference between this(...) and super(...)?
4. Where can a protected member be accessed?
5. What does final prevent for a field, method, and class?

# Lecture 14 in one view

## Superclass state is initialized first.

protected gives subclasses controlled access.  final places a boundary on reassignment or inheritance.

Bridge: every class hierarchy reaches Object; Lecture 16 explores Object and toString().

